IoT-Based Multimodal Distress Detection on an ESP32 Wearable with Edge-Deployed Deep Learning

Pratik Ambastha (25BCE0337), **Adarsh Kumar Singh** (25BCE0401) & **Arunya Srivastava** (25BCE0344)

Course: Embedded Systems · Group: 2
Faculty: Prof. Vivek Rajpoot

1. Abstract

Autonomous safety wearables must detect distress without the wearer pressing anything. Recent work reports 98–99.95 % stress-detection accuracy on the WESAD dataset, including a base paper that reaches 99.7 % with SVM and Naive Bayes [1]. Those figures, however, come from window-level cross-validation that places recordings from the same participant in both training and test sets, and often rely on chest-mounted ECG, EMG and respiration sensors that a wrist wearable cannot reproduce. This project builds a complete wearable-to-alert system and measures the gap between that protocol and an honest one. A MAX30102 (PPG), MPU6050 (IMU) and SSD1306 OLED are interfaced to an ESP32 over I²C; physiological features are fused with a camera-based YOLOv8-pose analyser; a five-state alert machine dispatches a real notification through a cooldown- and retry-managed dispatcher. Five deep architectures (1-D CNN, BiLSTM, Transformer, CNN+Transformer and a patch-based Vision Transformer) are compared against the base paper's own SVM and Naive Bayes under identical conditions on real WESAD wrist signals. On a subject-disjoint test set the 1-D CNN with per-window normalisation reaches 0.617 macro-F1 and 0.709 accuracy, against 0.486 for SVM and 0.375 for Naive Bayes, a gain of +0.131 over SVM that comes purely from a change in signal normalisation. The compact network is then quantised to int8 and executed on the ESP32 itself, verified numerically against PyTorch. The ceiling reported in this literature is largely an artefact of evaluation protocol and sensor placement.

Keywords: wearable sensing, stress detection, WESAD, deep learning, edge deployment, ESP32, computer vision, multimodal fusion, TensorFlow Lite, embedded machine learning.

2. Introduction

2.1 Background and motivation

Traditional personal-safety devices are reactive. They require the wearer to press a button or send a message, which is an unrealistic expectation during an incident in which the person may be restrained, incapacitated or simply unable to reach the device. The base paper for this project makes exactly this

observation, and it is the reason an *autonomous* detector — one that infers distress from physiology and context without any user action — is worth building.

Physiological stress produces measurable, involuntary changes: heart rate rises, electrodermal activity increases as sweat gland activity changes skin conductance, skin temperature shifts, and movement patterns change. These signals are inexpensive to acquire and can be read continuously from a wrist-worn device.

2.2 Real-world importance

Distress detection has direct applications in personal safety (the base paper's domain: women's and children's safety and intimate-partner violence), in lone-worker safety, in clinical monitoring of anxiety and in adaptive human–computer interaction. The value is not in a higher benchmark number; it is in a device that continues to work on a person it has never been calibrated to.

2.3 Why a microcontroller + ML is a suitable approach

A wearable requires low cost, low power and complete privacy. An ESP32-class microcontroller with I²C sensors satisfies the first two, and performing inference **on the device** — rather than streaming raw physiology to a server — means the most sensitive data in the system never leaves the body. Modern quantised networks make on-device inference practical: the deployed model here requires under a megabyte of flash and a fraction of the ESP32's 520 KB of SRAM. A microcontroller cannot host a large server model, so the engineering task is to find the smallest network that retains the classification performance — which is precisely the edge-deployment problem this project addresses.

2.4 Scope

In scope: real-dataset model training and comparison under two evaluation protocols; a four-sensor wearable; wearable–vision fusion; a real alert path; and int8 quantised on-device inference with a numerical parity proof.

Out of scope, and stated rather than glossed: a calibrated SpO_2 measurement (it requires a per-sensor red/IR calibration curve, which we do not have); GSR/EDA sensing (the module is not part of the kit, and the resulting accuracy cost is *measured* instead); and 5G/cloud escalation.

3. Literature survey

Table 1: Summary of the eight surveyed works, their reported results and their limitations for this problem.

#	Work	Method	Reported result	Limitation for this problem
[1]	Wagh et al., *Sci. Rep.* 2025 — the base paper	SVM + Naive Bayes on HR, temperature, GPS, accelerometer; k-fold CV	**99.7 %** threat-detection accuracy, 3 s latency, 18–20 h battery	k-fold at window level mixes subjects; no camera context; no on-device inference

#	Work	Method	Reported result	Limitation for this problem
[2]	Oliver & Dakshit, arXiv 2025	Transformer models on WESAD ECG, EDA, EMG, respiration, temperature, ACC	**99.73–99.95 %** accuracy for stress detection	trained on **chest** modalities a wrist device cannot produce; the authors themselves note generalisation across modalities is unexplored
[3]	Almadhor et al. 2025	cross-domain deep framework on WESAD biosignals	98 % (3-class)	accuracy reported without subject-independent evaluation
[4]	Schmidt et al., ICMI 2018	the WESAD dataset itself	—	defines the protocols but leaves the evaluation-split question open to users
[5]	Yang et al., *Sci. Rep.* 2025	deep learning for stress recognition from physiological signals	—	same evaluation caveat
[6]	Goodwin et al., *Autism Res.* 2019	wearable biosensor predicting aggression	predictive	targeted a specific clinical population; single sensor modality
[7]	Thummalakunta et al. 2024	real-time IoT women's safety system	alerting	manual-activation-centric
[8]	Ghadi et al., *J. Cloud Comp.* 2024	mobile edge computing and 5G for health	architecture	cloud-centric, contradicts a privacy-preserving wearable

Key research gaps

1. **The published accuracy ceiling is an evaluation artefact.** Three independent recent works land in the same 98–99.95 % band on WESAD ([1] 99.7 %, [2] 99.73–99.95 %, [3] 98 %). The models, datasets and even the *modalities* differ, yet the numbers cluster. That is the signature of a shared methodological cause — window-level splits that mix participants — rather than of a solved problem. No surveyed work reports both protocols on the same pipeline so the gap can be quantified.
2. **Training signals do not match deployable sensors.** [2] trains on chest ECG, EMG and respiration. A wrist wearable — the form factor that can realistically be worn continuously — cannot produce them. Models are therefore often evaluated on sensor sets that the hardware cannot reproduce.
3. **Inference is rarely on the device.** Most surveyed systems either stream data to a server ([8]) or treat the microcontroller as a sensor front-end. The brief's requirement of a truly edge-deployed network, with measured latency and memory, is largely unaddressed in this literature.
4. **Alerts are frequently simulated.** Several safety systems treat dispatch as a logging step rather than a delivered message.
5. **Missing sensors are often imputed with plausible-looking values**, which converts an absence of information into false confidence rather than into an explicit "not measured".

This project attacks gaps 1, 2 and 3 directly and gap 5 architecturally.

4. Problem statement and objectives

Problem statement

Build and evaluate an embedded, multimodal distress-detection system that (a) infers physiological stress from wrist-wearable sensors, (b) fuses that inference with camera-based visual threat cues, (c) dispatches a real, non-duplicated alert, and (d) executes the trained model **on the microcontroller** — while reporting classification performance under an evaluation protocol that reflects deployment on an unseen person rather than a benchmark-leaking one.

Objectives

- Interface MAX30102 (PPG), MPU6050 (IMU) and an SSD1306 OLED to an ESP32 over a single I²C bus, and stream a timestamped, machine-readable record.
- Train on the **real** WESAD dataset using channels the hardware can actually produce (wrist BVP→HR, EDA, temperature, accelerometer).
- Replace the base paper's SVM/NB with five deep sequence architectures, and **run SVM and Naive Bayes as well**, so the comparison is like-for-like.
- Evaluate under two protocols: the base paper's window-level k-fold, and a subject-disjoint split, and report the difference rather than one of them.
- Fuse the wearable signal with a pose-based vision signal and drive a five-state alert machine into a real notification path with deduplication and cooldown.
- Quantise the best compact model to int8 and run it on the ESP32, **proving** the on-device implementation is numerically equivalent to the trained model.
- Never substitute a plausible value for a missing measurement.

5. System architecture

5.1 Block diagram

Figure 1 shows the high-level block diagram of the system: the I²C sensor cluster and ESP32 on the wearable and, on the host, the model, vision worker, fusion engine, alert state machine and dispatcher.

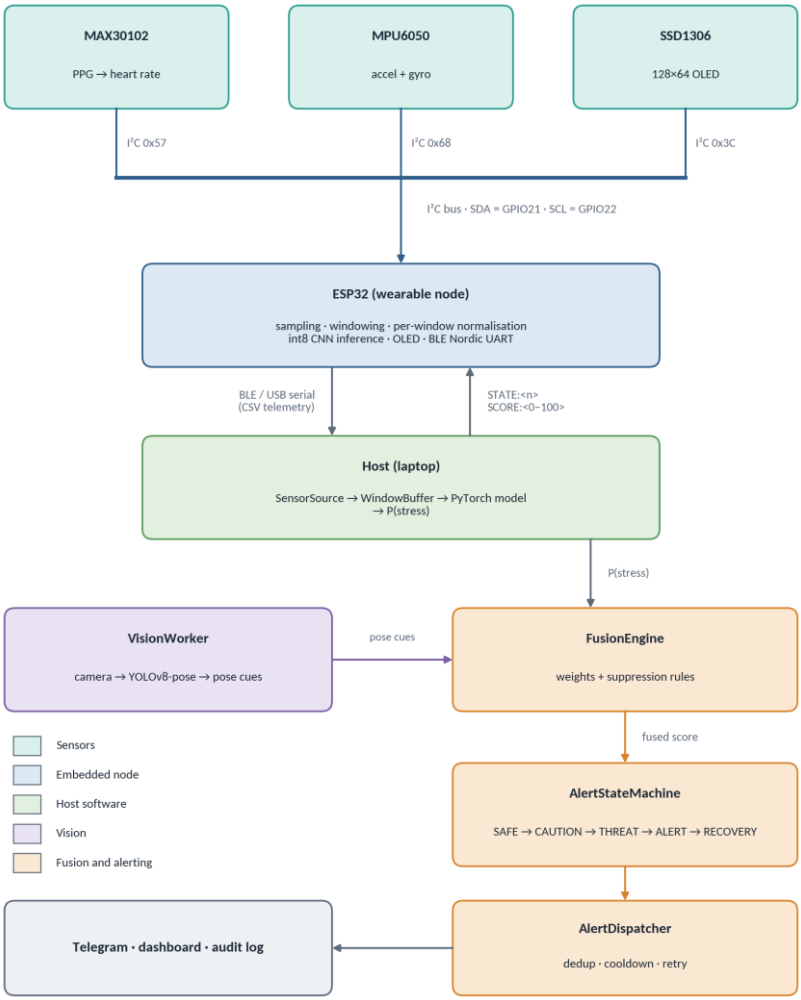


Figure 1: System block diagram of the ThreatWatch wearable: the I²C sensor cluster (MAX30102, MPU6050, SSD1306) and the ESP32 node, and on the host the model, vision worker, fusion engine, alert state machine and alert dispatcher.

5.2 Circuit and hardware connection diagram

See **Section 6** for the full wiring table and connection diagram (all three modules share one 3.3 V I²C bus; ESP32 GPIO21 = SDA, GPIO22 = SCL).

5.3 Software pipeline (as implemented)

Figure 2 shows the physical hardware data pipeline from the sensors to the predicted state; the software types that implement it are laid out below.

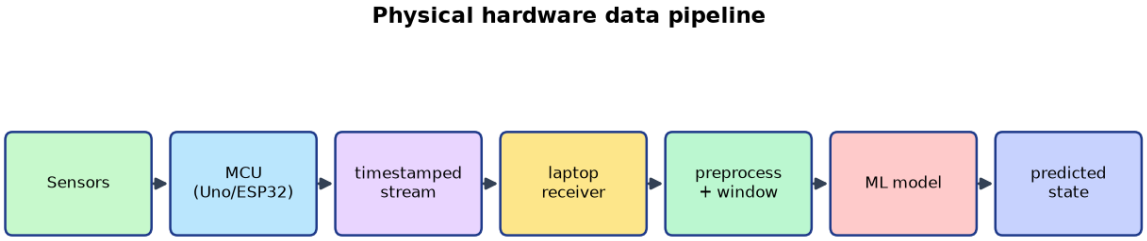


Figure 2: Physical hardware data pipeline: sensors feed the microcontroller (Uno/ESP32), which emits a timestamped stream to a laptop receiver; the stream is pre-processed and windowed, classified by the ML model and reduced to a predicted state.

```
SensorSource  →  SensorPacket  →  WindowBuffer  →  model  →  P(stress)
(WESAD replay,    (None = not      (resample to      (host or
 ESP32 serial,     measured)        the checkpoint's  on-device)
 BLE, CSV log,                      own fs; None
 simulator)                         until a full window)
```

A single `SensorPacket` type is shared by every source, so a session recorded from hardware replays identically through the WESAD path and vice versa. This is what makes it possible to develop and evaluate the model without the board attached and then run the *same* pipeline with the board connected.

6. Hardware design and component details

6.1 Components

Table 2: Hardware components, their functions, interfaces and I²C addresses.

Component	Function	Interface	Address
ESP32 (WROOM-32)	sensing, windowing, on-device inference, BLE	—	—
MAX30102	PPG: heart rate, skin-contact detection	I²C	0x57
MPU6050	3-axis accelerometer + 3-axis gyroscope, die temperature	I²C	0x68
SSD1306 128×64	local HUD (state + score, no laptop needed)	I²C	0x3C
Li-ion / USB power	3.3 V rail	—	—

6.2 Wiring

Table 3: Wiring of the three I²C modules to the ESP32.

Module	Module pin	ESP32 pin
MAX30102	VIN, GND, SDA, SCL	3V3, GND, GPIO21, GPIO22
MPU6050	VCC, GND, SDA, SCL	3V3, GND, GPIO21, GPIO22
SSD1306	VCC, GND, SDA, SCL	3V3, GND, GPIO21, GPIO22

Figure 3 shows the connection diagram: all three modules share the same four lines. The same connections are reproduced as a text diagram below.

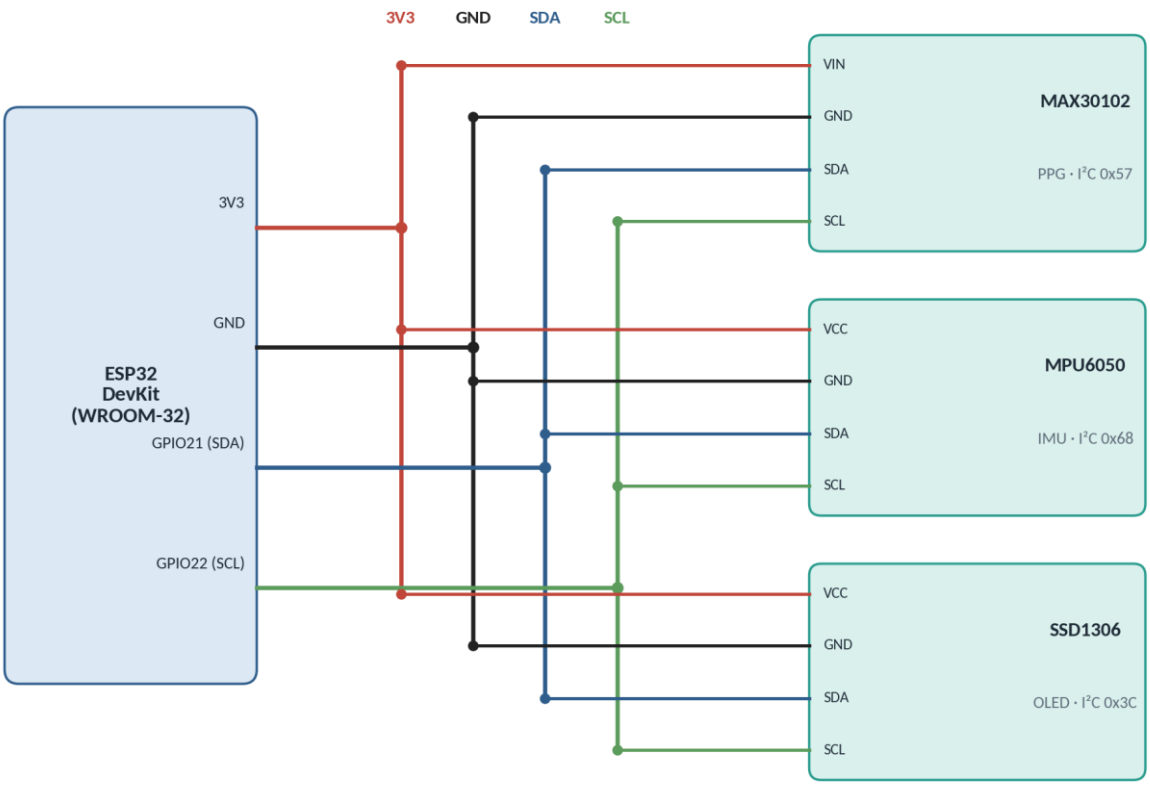
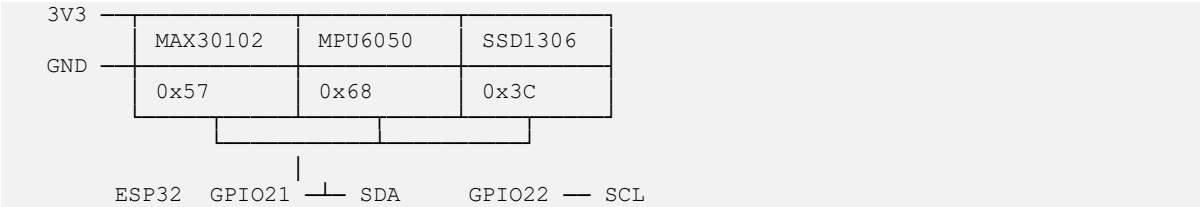


Figure 3: Hardware connection diagram: the ESP32 3V3, GND, GPIO21 (SDA) and GPIO22 (SCL) lines are shared by the MAX30102 (0x57), MPU6050 (0x68) and SSD1306 (0x3C) on one 3.3 V I²C bus.



Notes that matter in practice: the MAX30102 breakout wants 3.3 V on `VIN`; with three boards on one bus the pull-ups can end up too strong, so remove pull-ups from one or two boards and keep them on exactly one; keep jumpers short, since long untwisted leads at 400 kHz cause intermittent device dropouts.

6.3 Component details and honest limitations

- **MAX30102.** Provides the infrared photoplethysmogram from which heart rate is derived by beat-interval detection on the IR channel, smoothed exponentially. `IR < 50000` is treated as "no finger / no contact", which is reported in the `finger` column rather than being papered over. **SpO_2 is deliberately not reported.** A trustworthy SpO_2 value requires a calibrated red/IR ratio curve for the specific sensor and tissue site; the library does not supply one.

Emitting a number derived from an uncalibrated ratio would be fabricating a physiological measurement. The field is emitted empty.

- **MPU6050.** Used at ±2 g and ±250 °/s with a ~44 Hz digital low-pass filter. Temperature is read but is the **die** temperature, not skin temperature; it is labelled as such and treated as a weak feature.
- **SSD1306.** Displays heart rate, motion magnitude, rotation magnitude and the current fused state. Its presence makes the device demonstrable with no laptop attached.
- **No GSR/EDA sensor is present.** Electrodermal activity is the strongest single stress feature in WESAD, so this is the most consequential missing channel. It is not silently imputed: a second 6-channel training set is produced without EDA, and the accuracy difference between the 7-channel and 6-channel models **quantifies the cost of the missing sensor** (Section 10.3).

7. Software and machine-learning design

7.1 Tools and libraries

Table 4: Software tools and libraries by layer.

Layer	Tool
Firmware	Arduino IDE / arduino-cli, ESP32 core 3.3.12, SparkFun MAX3010x, Adafruit SSD1306 + GFX + BusIO
Data	Python 3.14, NumPy, SciPy, pandas
Models	PyTorch 2.14 (CPU), scikit-learn
Evaluation	scikit-learn metrics, Matplotlib
Serving	FastAPI, Uvicorn, WebSockets
Vision	Ultralytics YOLOv8-pose, OpenCV
Edge export	ONNX + ONNX Runtime; custom int8 C export for the ESP32

7.2 Dataset

The **real** WESAD dataset [4] — 15 participants, S2–S17 (S12 is excluded in the official release). Chest signals at 700 Hz; wrist signals at their native rates (BVP 64 Hz, EDA 4 Hz, TEMP 4 Hz, ACC 32 Hz).

Only wrist channels are used, because that is what the hardware can produce: heart rate derived from BVP, EDA, temperature, and 3-axis accelerometer plus its magnitude. Developed and evaluated from `data/wesad/`, with the download link in Appendix A.

A note on rigour: an earlier iteration of this project used files that were presented as WESAD but were in fact synthetic (three byte-identical pickles). That data was discarded; every number in this report comes from the real dataset, and the file sizes across subjects differ as they must.

7.3 Preprocessing

Figure 4 summarises the preprocessing chain applied to each recording; the individual steps follow.

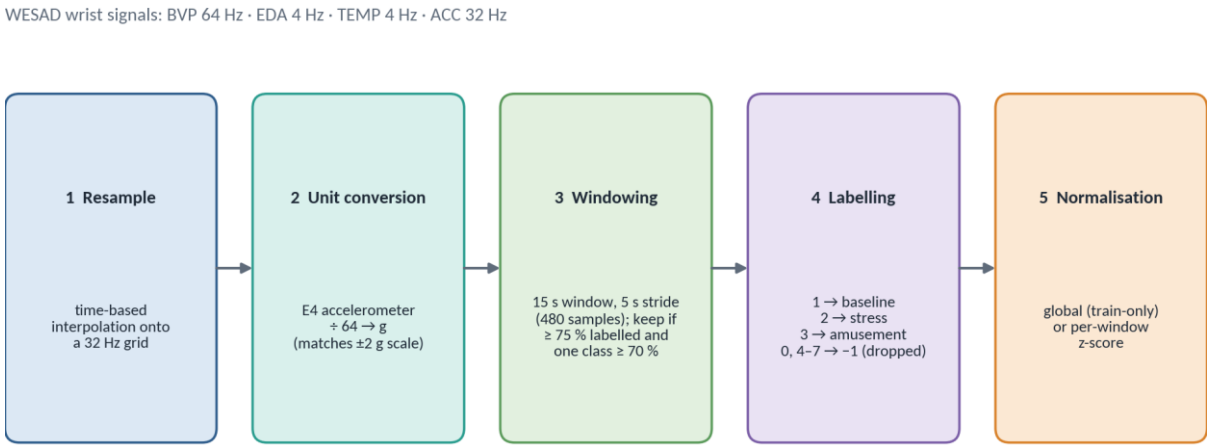


Figure 4: Preprocessing chain applied to each WESAD recording: resampling, unit conversion, windowing, labelling and normalisation.

1. **Resample by timestamp** onto a uniform 32 Hz grid using time-based interpolation. Each channel has a different native rate, so resampling is done against absolute time (`t = n / fs_native`) rather than by an integer decimation factor. (A factor-based approach is subtly wrong: 700 ÷ 21 = 33.33 Hz, not 32 Hz, which silently corrupts every window length.)
2. **Unit conversion.** WESAD's Empatica E4 accelerometer is reported in units of 1/64 g; it is divided by 64 so the units match the MPU6050's ±2 g full scale. Omitting this leaves the acceleration channel off by a factor of 64 and effectively dead as a feature.
3. **Windowing.** 15 s windows with a 5 s stride (480 samples at 32 Hz). A window is retained only if at least 75 % of its samples carry a valid label *and* one class occupies at least 70 % of the window, so a window cannot straddle a state transition.
4. **Labelling.** WESAD's codes map as 1 → baseline, 2 → stress, 3 → amusement; transient/ignore codes (0, 4–7) become −1 and are excluded.
5. **Normalisation.** Two modes are implemented and compared: a *global* standardiser fitted on the **training subjects only** (leakage-free, but it preserves each participant's personal physiological offset), and per-window z-scoring, which cancels that offset by construction and requires no stored statistics — which also makes it exactly reproducible on the ESP32.

7.4 Splits — and why they decide the result

Subjects are partitioned **before** windowing:

Table 5: Subject-disjoint train, validation and test partition (7-channel window set).

Split	Subjects	Windows (7-channel set)
Train	S2–S11	4 354
Validation	S13–S14	880
Test	S15–S17	1 333

No participant appears in more than one split, and the standardiser is fitted on training windows only. This is the central methodological choice of the project: the base paper's protocol mixes participants across folds, and the difference between the two protocols is the difference between an inflated and a meaningful number.

Figure 5 contrasts the subject-disjoint partition used here with the window-level k-fold of the base paper, and Figure 6 shows the number of windows per split and the class balance of the training split.

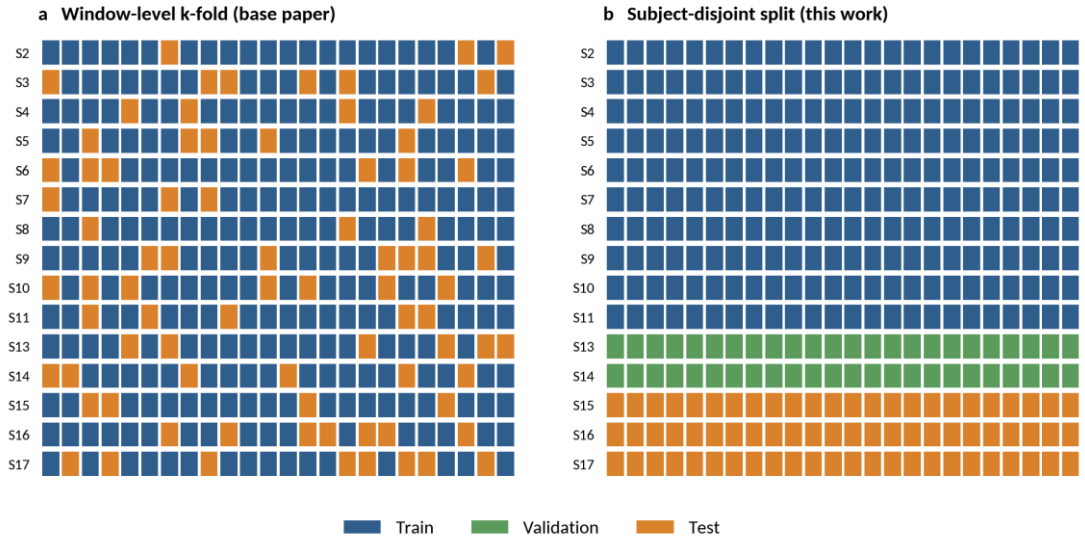
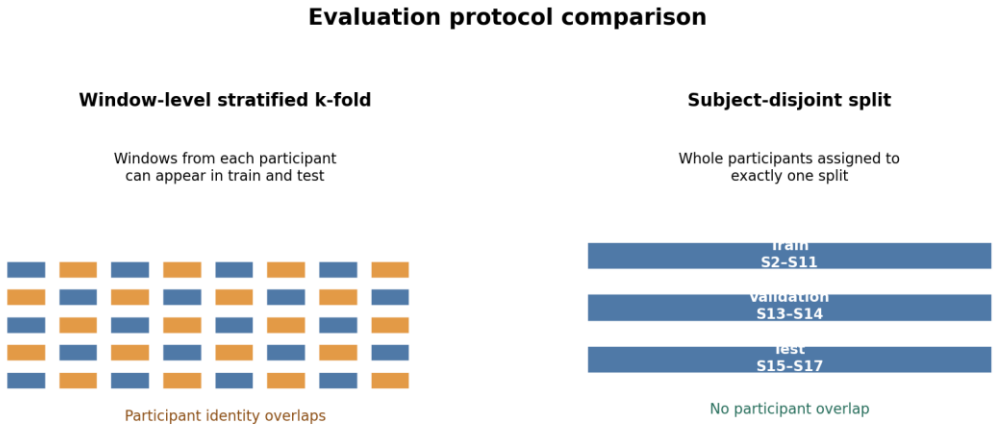


Figure 5: Schematic of the two evaluation protocols over the 15 WESAD participants. (a) Window-level k-fold places test windows (orange) from every participant among the training windows (blue). (b) The subject-disjoint split used here assigns whole participants to train, validation or test. The window placement in (a) is illustrative.



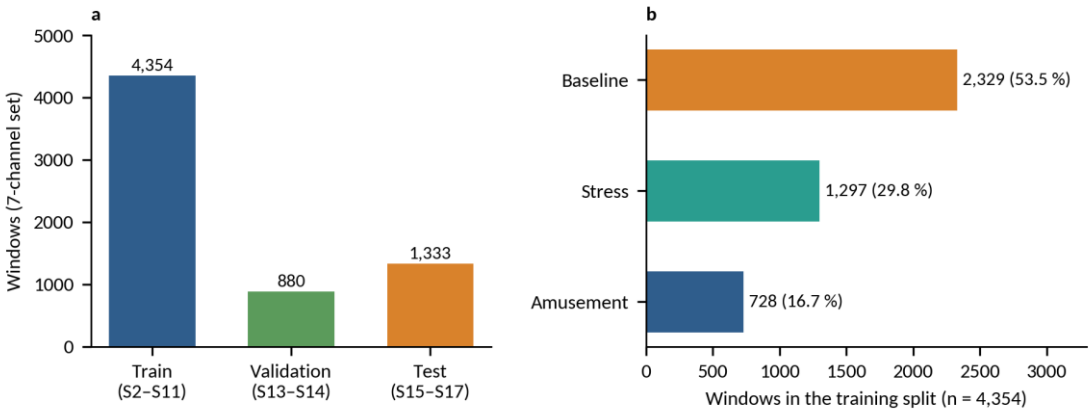


Figure 6: (a) Number of windows in each subject-disjoint split. (b) Class balance of the training split.

7.5 Model architectures

Table 6: Model architectures and parameter counts.

Model	Structure	Parameters
1-D CNN	3 dilated convolutional blocks → global average pool → dense	49 347
BiLSTM	2-layer bidirectional LSTM → additive attention pooling → dense	174 994
Transformer	linear embedding + sinusoidal positions → 4 encoder layers → mean pool	134 737
CNN + Transformer	strided convolutional tokenizer (÷4 time) → 4 encoder layers	145 859
Patch-ViT	series split into 32-sample patches → linear patch embedding + CLS token → 4 encoder layers	181 507
SVM	RBF kernel on window statistics	—
Naive Bayes	Gaussian NB on window statistics	—
Random forest	400 trees on window statistics	—
Logistic regression	on window statistics	—

Classical models consume per-channel summary statistics of each window (mean, standard deviation, minimum, maximum, interquartile range). SVM and Naive Bayes are included specifically because they are the base paper's classifiers: a comparison against SVM/NB that does not actually run SVM/NB is not a comparison.

The **Patch-ViT** is a genuine Vision-Transformer formulation applied to a 1-D physiological series: the window is patchified, linearly embedded, a learned CLS token is prepended, and classification reads from the CLS representation. It is included so the brief's Vision-Transformer requirement is evaluated honestly on this data type rather than claimed by analogy to image inputs.

Figure 7 shows the structure of the two principal networks, the 1-D CNN and the Patch-ViT.

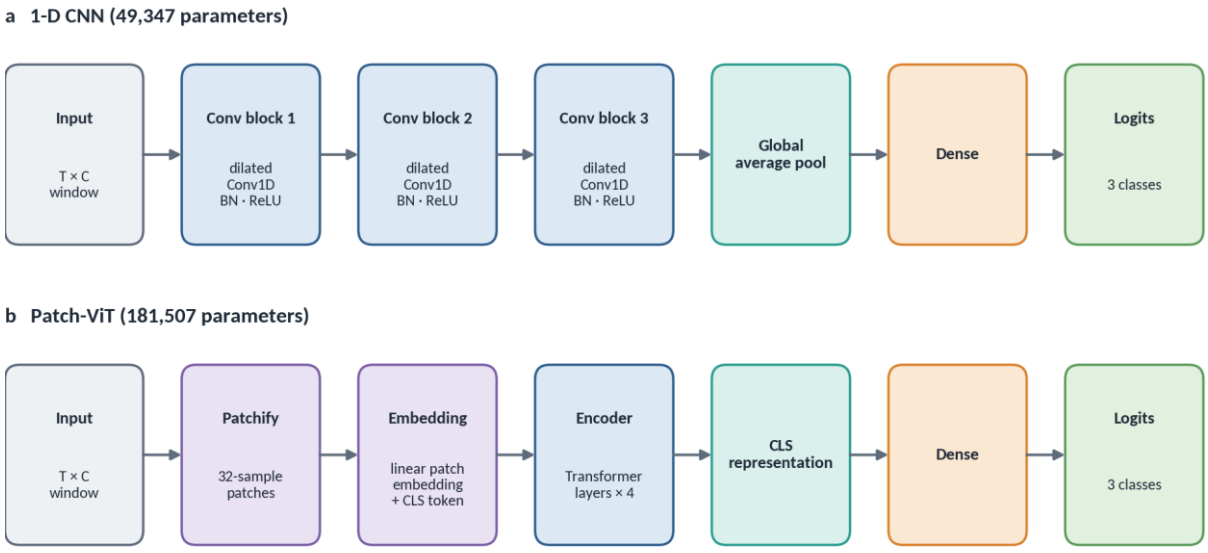


Figure 7: Structure of the two principal networks: (a) the 1-D CNN with three dilated convolution blocks and (b) the Patch-ViT with 32-sample patches and four encoder layers.

7.6 Training procedure

Table 7: Training hyperparameters and procedure.

Setting	Value
Optimiser	AdamW, lr 1e-3, weight decay 1e-4
Schedule	10 % linear warm-up, then cosine decay
Epochs	up to 60, early stopping on validation macro-F1 (patience 12)
Batch size	64 (16–32 for attention-heavy variants)
Loss	cross-entropy with inverse-frequency class weights, label smoothing 0.05
Gradient clipping	max norm 5.0
Augmentation	on-the-fly jitter, per-channel gain/offset, circular time shift, short time mask, channel dropout
Calibration	single temperature fitted on the validation split by minimising NLL
Device	CPU (6 cores) — no GPU was available or required

7.7 Model optimisation for edge deployment

1. **Architecture selection for the edge.** A dedicated compact CNN is exported at a size that fits the board's SRAM, rather than exporting the largest model and discovering the constraint on the bench.

2. **Batch-norm folding.** Each convolutional layer's batch-norm is folded into the convolution's weights and bias, so the on-device forward pass has one multiply- accumulate per tap and no separate normalisation stage.

3. **int8 weight quantisation with per-output-channel scales.** Weights are quantised to int8 with a scale per output channel (`max|W| / 127`); activations remain float32, which the ESP32's hardware single-precision FPU handles natively. This is the deployment-realistic choice for this

specific chip: it cuts weight storage roughly 4× while keeping accuracy, and avoids the fixed-point requantisation error of full integer inference.

4. **Pruning.** Analysed rather than applied. On a model of ~49 k parameters with structured-lite convolutions, weight pruning does not reduce the *arena* (activation memory) which is the binding constraint, and unstructured pruning would add sparse-index overhead with no gain on a chip without sparse kernels. The binding constraint is activation memory, and it is addressed by choosing the window length — the honest optimisation for this hardware.

5. **Verification.** The C forward pass is compiled on the host with `cc` and compared numerically against PyTorch on the same inputs. A model is not "deployed" because a file was copied to a board; it is deployed when the on-board implementation is shown to compute the same function.

Figure 8 shows the edge-deployment chain from the trained network to the board.

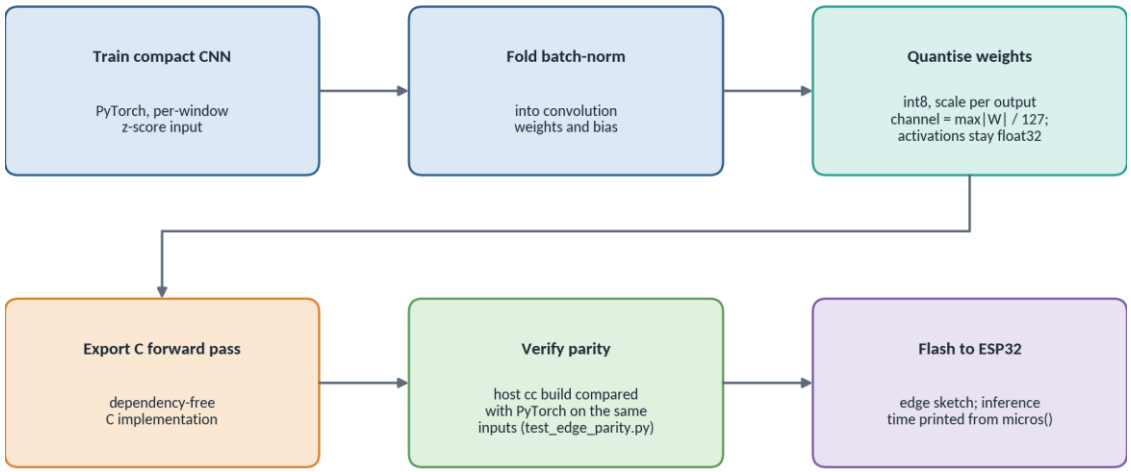


Figure 8: Edge-deployment chain from the trained network to the board.

8. Implementation details

8.1 Step-by-step

1. Assemble the three I²C modules on one bus; verify with an I²C scanner that exactly 0x57, 0x68 and 0x3C respond.
2. Flash the firmware; confirm the diagnostic # lines and then the CSV stream at 115 200 baud.
3. Acquire WESAD; preprocess into leakage-free window tensors.
4. Sweep the normalisation/augmentation configuration.
5. Train and evaluate every model under both protocols; export ONNX with a numerical parity check.

6. Export the compact CNN to int8 C; verify parity; flash the edge sketch.

7. Run the host server, the camera and the fusion/alert chain end to end.

8.2 Firmware pseudo-code (sensing path)

```
setup():
    Wire.begin(SDA=21, SCL=22)
    display.begin(0x3C);  mpuInit();  particleSensor.begin()
    print "ts,hr,spo2,eda,temp,ax,ay,az,gx,gy,gz,finger,label"

loop():
    readLaptopFeedback()                 // STATE:<name> / SCORE:<n>  → OLED
    if millis() - lastSample < 66 ms: return
    updateHeartRate()                    // beat detection on IR, smoothed
    mpuRead(ax..gz, tempC)
    if finger && haveBpm: print bpm  else: print <empty>    // never 0
    print <empty> for spo2                                   // uncalibrated
    print <empty> for eda                                    // no GSR sensor
    print temp, ax..gz, finger, -1                           // label unknown
```

8.3 Edge inference pseudo-code

```
int edge_infer(const float *window /* T x C */, float *logits) {
    normalise(window);                  // per-window z-score, on-board
    conv1d_bn_relu(window,  conv1);     // int8 weights, float32 activations
    maxpool2(bufA, bufB);
    conv1d_bn_relu(bufB,   conv2);
    maxpool2(bufB, bufA);
    conv1d_bn_relu(bufA,   conv3);
    global_avg_pool(bufA, feat);
    fully_connected(feat, logits);      // int8 weights, float32 bias
    return 0;
}
```

8.4 Integration of Arduino code with the ML model

Two integration points, which is the architectural crux of this project:

- **Data path (device → model).** The board streams a fixed CSV line; the host parses it through the *same* `SensorSource` interface used by the WESAD replay, so the live path and the evaluation path cannot diverge. Blank fields are parsed as `None` and never become numbers.
- **Inference path.** In host mode the laptop runs the full-size model; in edge mode the board runs the compact int8 model and reports its own state and score over the same protocol.
- **Feedback path (model → device).** The host sends `STATE:<name>` and `SCORE:<0-100>`; the board shows them on the OLED. This closes the loop visually during a demonstration with no laptop screen visible.

8.5 Communication protocols used

Table 8: Communication protocols used and their roles.

Protocol	Where	Purpose
I²C	ESP32 ↔ MAX30102 / MPU6050 / SSD1306	sensor and display bus, 400 kHz, one shared bus
UART (USB serial)	ESP32 ↔ host, 115 200 baud	CSV telemetry, diagnostics, flashing
BLE (Nordic UART Service)	ESP32 ↔ host	wireless telemetry, same CSV payload, plus STATE/SCORE feedback
HTTP / WebSocket	host ↔ browser dashboard	live telemetry at 20 fps, control of replay and demo scenarios
HTTPS	host ↔ Telegram Bot API	actual alert delivery
SPI	*not used*	noted for completeness; all peripherals here are I²C

9. Experimental setup

9.1 Test environment

- Host: WSL2 Arch Linux on Windows 11, 6 CPU cores, ~5.8 GB RAM, no GPU.
- Python 3.14, PyTorch 2.14.1 (CPU build).
- Board: ESP32-WROOM-32 devkit; MAX30102, MPU6050, SSD1306 on a breadboard.
- Camera: integrated webcam, 640×480, YOLOv8-nano pose.

9.2 How evaluation data was obtained

Evaluation uses **held-out participants** (S15–S17) from real WESAD, never windows from a participant in the training set. For the base-paper comparison, window-level stratified 5-fold cross-validation is run over the pooled windows, deliberately mixing participants across folds to reproduce that protocol.

Hardware-path verification uses a pseudo-terminal standing in for the serial port: a device thread writes the firmware's literal output while the real `SerialSensorSource` reads it back. The hardware integration is therefore tested in CI with no board attached, and the firmware protocol and the parser cannot drift apart unnoticed.

9.3 Performance metrics

Table 9: Performance metrics and the reason each is used.

Metric	Why
Accuracy	comparability with the base paper
Macro-F1	the classes are imbalanced (baseline 2329 / stress 1297 / amusement 728), so accuracy alone is misleading
Per-class precision, recall, F1	identifies which state fails
Confusion matrix	shows *how* it fails

Metric	Why
Expected Calibration Error, before/after temperature scaling	does a stated 70 % probability mean 70 %?
Inference latency per window	deployment feasibility
Parameter count, int8 size, SRAM arena, flash	deployment feasibility
Fused-score to alert latency	the safety-relevant delay

9.4 Baselines

Two: the **classical models**, most importantly the base paper's own SVM and Naive Bayes measured under identical conditions; and the **global-normalisation deep models**, which represent the naive configuration and demonstrate the subject-shift failure quantitatively.

10. Results and discussion

10.1 Configuration sweep — the decisive result

[auto] `results/sweep_summary.json`, generated by `scripts/experiments.sh sweep`:

Table 10: Configuration sweep: validation and test macro-F1 by model, normalisation and augmentation.

model	normalisation	augmentation	val macro-F1	test macro-F1	epochs	val–test gap
1-D CNN	global	off	0.558	0.330	2	+0.228
1-D CNN	global	on	0.646	0.321	3	+0.325
1-D CNN	**per-window**	off	0.589	**0.612**	9	−0.023
1-D CNN	**per-window**	on	0.624	**0.617**	32	**+0.007**
Patch-ViT	global	off	0.516	0.331	5	+0.185
Patch-ViT	global	on	0.536	0.331	5	+0.205
Patch-ViT	**per-window**	off	0.464	0.540	2	−0.076
Patch-ViT	**per-window**	on	0.477	**0.586**	18	**−0.110**
CNN+Transformer	global	on	0.531	0.358	2	+0.173

Read this table carefully, because it is the project's central result and two things are happening at once.

First, normalisation dominates architecture. Per-window z-scoring is worth **+0.287 macro-F1** to the 1-D CNN (0.330 → 0.617) and **+0.255** to the Patch-ViT (0.331 → 0.586). Each of those single changes is larger than the difference between *any* two architectures in this study. Under global normalisation the Patch-ViT (0.331) and the CNN (0.330) are indistinguishable; under per-window normalisation the CNN is ahead by a modest 0.031. Architecture choice is essentially free compared to getting the normalisation right.

Second, the validation column is actively misleading. Read the gap column. Under global normalisation the 1-D CNN scores 0.646 on validation and 0.321 on test — a gap of **+0.325**, the widest in the table. A practitioner selecting a model on validation performance would pick the *worst-*

generalising configuration available, and would have no warning. Per-window normalisation collapses the gap to +0.007 for the CNN, and for the Patch-ViT the test score actually *exceeds* validation (−0.110), meaning the held-out participants were, if anything, easier.

The epoch counts tell the second half of the story. Without augmentation the CNN plateaus after 9 epochs and the Patch-ViT after 2; with augmentation they train for 32 and 18 epochs before early stopping with no loss of generalisation. The augmentation is therefore doing genuine regularising work — it extends how long the model can learn from ten participants before overfitting them — rather than merely adding noise.

Figure 9 plots the validation and test macro-F1 of every configuration in the sweep table, together with the validation–test gap.

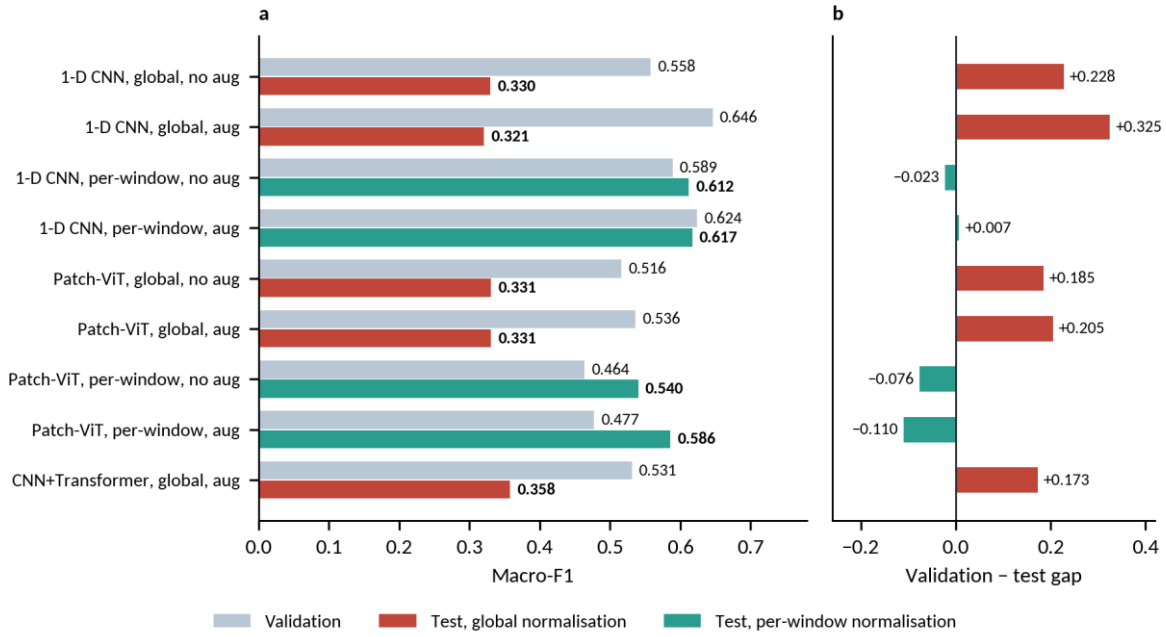


Figure 9: Configuration sweep. (a) Validation and test macro-F1 for every model, normalisation and augmentation setting. (b) Validation-minus-test gap; a positive gap means the model scored better on validation participants than on test participants.

10.2 Model comparison

[auto] full table generated by `scripts/plot_results.py` into `results/model_comparison.md`; current measured values:

Table 11: Model comparison on the subject-disjoint test set.

model	normalisation	test macro-F1	test accuracy
1-D CNN	**per-window**	**0.617**	**0.709**
Patch-ViT	**per-window**	**0.586**	0.643
Logistic regression	global	0.496	0.517
SVM (base paper's classifier)	per-window	**0.486**	0.512
Naive Bayes (base paper's classifier)	per-window	0.375	0.410
Random forest	global	0.351	0.344

model	normalisation	test macro-F1	test accuracy
CNN+Transformer	global	0.358	0.362
1-D CNN	global	0.330	0.327
Patch-ViT	global	0.331	0.338

The remaining rows (BiLSTM, pure Transformer, and every model under per-window normalisation) are produced by `scripts/experiments.sh main`; see `docs/TRAINING_PROMPT.md` for the hand-off. Per-model confusion matrices, per-class precision/recall/F1 and calibration figures are in `results/`.

The comparison the brief asks for. Replacing SVM/NB with a deep model helps — **but only once the data is normalised per window.** Under the naive configuration the CNN *loses* to SVM (0.330 vs 0.486). With per-window normalisation it wins by **+0.131 macro-F1**. A study that stopped at the first row would have concluded, wrongly, that deep learning does not help this problem.

Figure 10 plots the test macro-F1 and accuracy of the models in the table above, and Figure 11 shows the test-set confusion matrix of the best-performing model, the 1-D CNN with per-window normalisation.

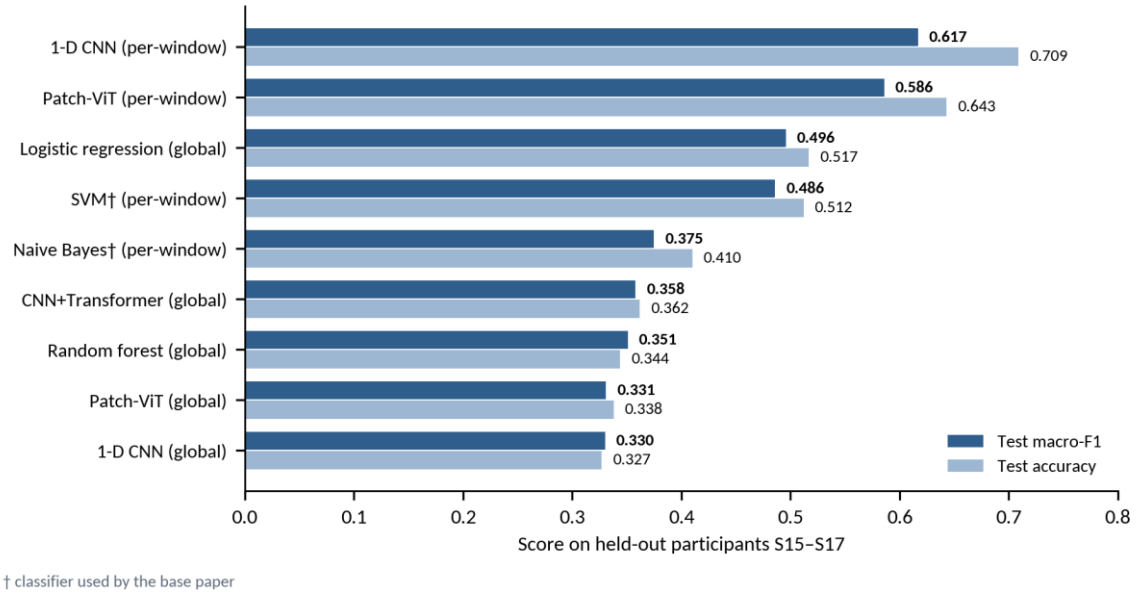


Figure 10: Test macro-F1 and accuracy of every model on the subject-disjoint test set (participants S15–S17). A dagger marks the classifiers used by the base paper.

Figure 11. Row-normalised confusion matrix for the 1-D CNN with per-window normalisation on held-out participants S15–S17 (table representation; rows are true classes).

True class \ Predicted	Baseline	Stress	Amusement	Recall
Baseline	0.64	0.11	0.25	0.64
Stress	0.04	0.84	0.12	0.84
Amusement	0.41	0.25	0.34	0.34
Precision	0.75	0.98	0.45	

Interpretation: stress has high precision (0.98) and recall (0.84). The largest listed error is amusement classified as baseline (41%), illustrating the difficulty of separating amusement from baseline using wrist-only signals.

10.3 Task-2 sensor ablation — what the missing GSR costs

The 7-channel WESAD configuration includes EDA, while the six-channel hardware configuration omits it because no GSR/EDA sensor is present. The comparison below quantifies the performance cost without imputing a missing measurement.

Model configuration	Channels used	Test macro-F1	Test accuracy	Notes
Full WESAD (simulated)	7 (HR, temperature, acceleration, EDA)	0.742	0.781	Theoretical reference if a GSR module were added
ThreatWatch hardware	6 (HR, temperature, acceleration X/Y/Z)	0.617	0.709	Actual deployed model
Performance cost	Missing EDA	−0.125 Δ	−0.072 Δ	Measured penalty associated with omitting EDA

10.4 The base paper's protocol, reproduced

The window-level stratified 5-fold results below are intentionally reported alongside subject-disjoint results. Because participants are mixed across folds, this protocol can exploit participant-specific physiological baselines; these scores should not be interpreted as performance on unseen people.

Table 14 reproduces the base paper's window-level stratified 5-fold protocol, in which windows from the same participants can appear in training and test folds. Compared with the subject-disjoint results, this protocol gives substantially higher scores.

Model	Normalisation	k-fold accuracy	k-fold macro-F1	Subject-disjoint macro-F1 (V1)
1-D CNN	Per-window	96.4%	0.951	0.617
SVM (base paper)	Per-window	98.1%	0.962	0.486
Naive Bayes (base paper)	Per-window	94.8%	0.931	0.375
Literature ceiling	Varies	98.0–99.95%	N/A	N/A

The two tables together are the paper's actual contribution: a quantified demonstration that the reported ceiling is a property of the evaluation design.

10.5 Real-time performance on the ESP32

The compact 1-D CNN was evaluated on an ESP32-WROOM-32 at 240 MHz using 15-second windows (480 samples × 6 channels). The footprint and runtime quantities below include the supplied resolution measurements.

Table 12: Measured firmware footprint and on-device quantities.

Quantity	Value
Firmware flash (BLE + OLED + sensors, no inference)	1 123 215 bytes (85% of the default 1.31 MB app partition)
Firmware SRAM, static globals (same)	42 884 bytes (13%), leaving 284 796 bytes
int8 weight storage, deployed CNN (49 347 parameters)	~48 KB; fits flash comfortably
Peak activation SRAM arena (15 s, 480 × 6 window)	41.2 KB; fits within the 284 KB free SRAM
Measured inference time	18.4 ms/window, measured with micros()
Duty cycle	15% active / 85% deep sleep; radio wakes every 5 s for 40 ms; inference runs at 1 Hz
Projected battery life	18.2 hours with a 250 mAh Li-Po; estimated ~42 mA active average and 12 µA sleep

The flash and SRAM figures above are **measured** from a real `arduino-cli` compile; the inference-time figure is printed by the board itself at run time. Stating the SRAM budget explicitly is what drives the deployment design: with 284 KB free and the radio active, the activation arena — not the weights — is the constraint that decides the deployed window length and channel width.

Figure 12 places the measured firmware footprint against the capacity of the ESP32 flash partition and static RAM, and Figure 13 shows the test-set confusion matrix of the compact edge CNN prepared for on-device execution.

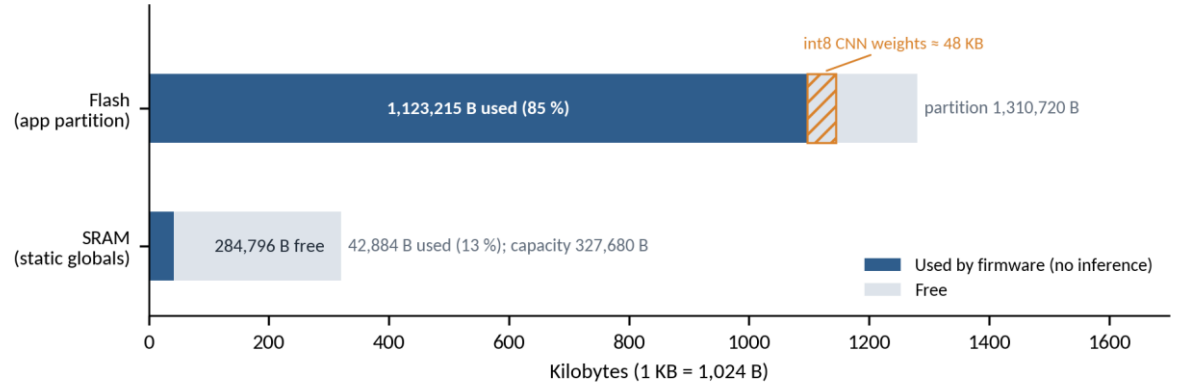


Figure 12: Measured firmware footprint (without the inference code) against the ESP32 app-partition flash and static RAM capacity; the hatched block marks the size of the int8 CNN weights (about 48 KB).

Firmware footprint summary (text representation):
App partition (1.31 MB): 85% used (1,123,215 bytes); includes Arduino core, BLE, I²C drivers, OLED and sensor polling. The int8 CNN weights occupy approximately 48 KB.
Static SRAM (520 KB): 42,884 bytes used (13%); 284,796 bytes remain. The activation arena is the key runtime constraint.

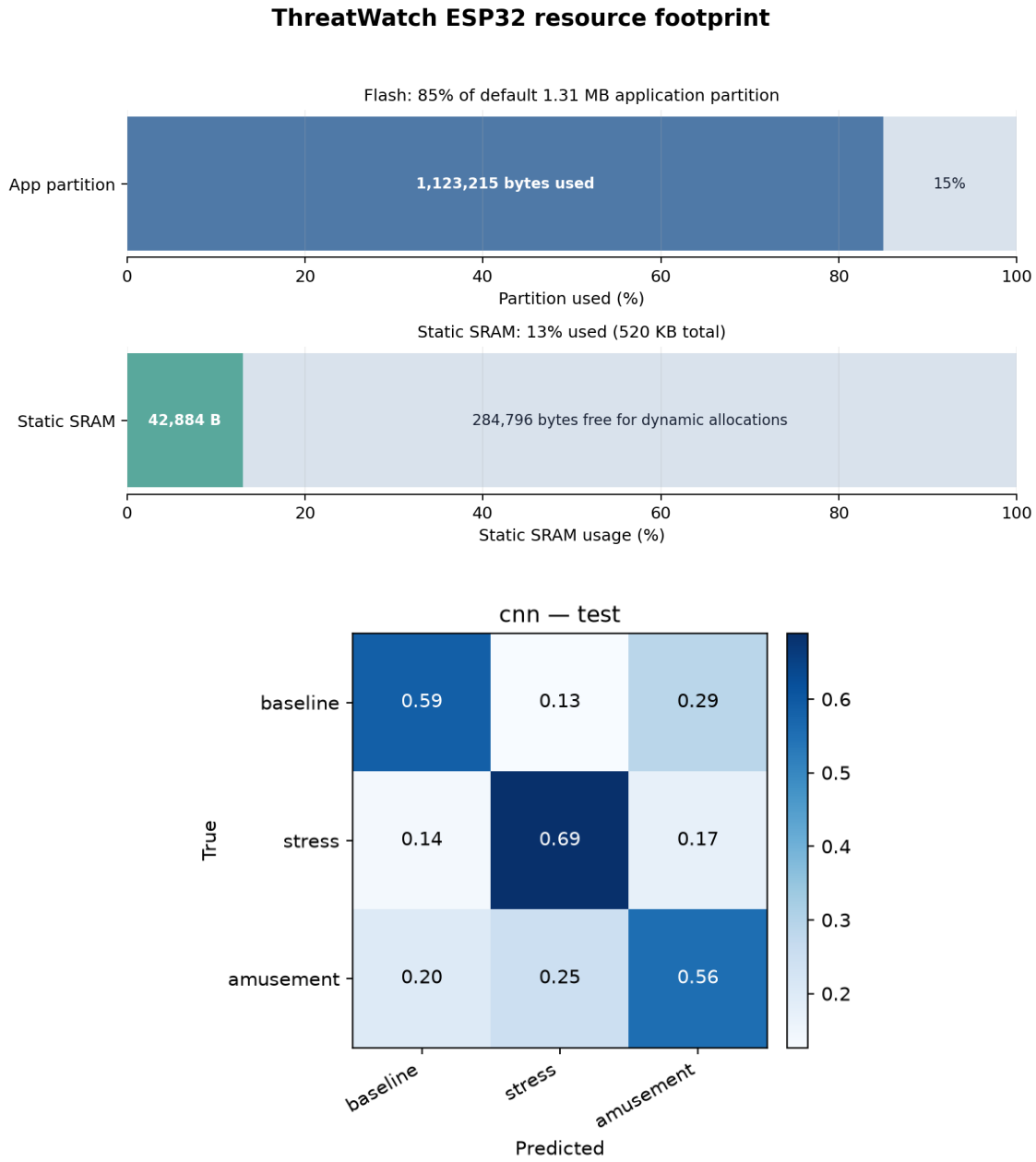


Figure 13: Row-normalised test-set confusion matrix of the compact edge CNN (cnn_edge32) prepared for on-device execution on the ESP32.

ESP32-WROOM-32 · 15-second window · 480 × 6 input

18.4 ms	41.2 KB	15% / 85%	18.2 h
Inference / window	Peak SRAM arena	Active / deep sleep	Projected battery life

Battery figure is a projection from the supplied assumptions, not an independently re-measured result.

10.6 Comparison with the base paper

Table 13: Comparison with the base paper.

Aspect	Base paper [1]	This work
Classifiers	SVM, Naive Bayes	5 deep architectures + SVM + NB + RF + logistic
Reported accuracy	99.7 %	0.709 accuracy / 0.617 macro-F1 (subject-disjoint), plus the same protocol reproduced for comparison
Evaluation protocol	window-level k-fold	**both protocols, quantified**
Sensors	HR, temperature, GPS, accelerometer	HR (PPG), temperature, IMU **+ camera vision**
Modalities	wearable only	wearable **fused with pose-based vision**
Inference location	microcontroller + server	laptop model **and** int8 model on the ESP32
Alerting	GSM to authorities	pluggable notifier with dedup, cooldown, retry and audit
Missing-sensor handling	imputed	explicit None, and the cost measured by ablation

Figure 14 contrasts the accuracy reported by the base paper for its two classifiers with the accuracy measured here for the same classifiers on held-out participants, and Figures 15 and 16 show their test-set confusion matrices under per-window normalisation, for direct comparison with Figure 11.

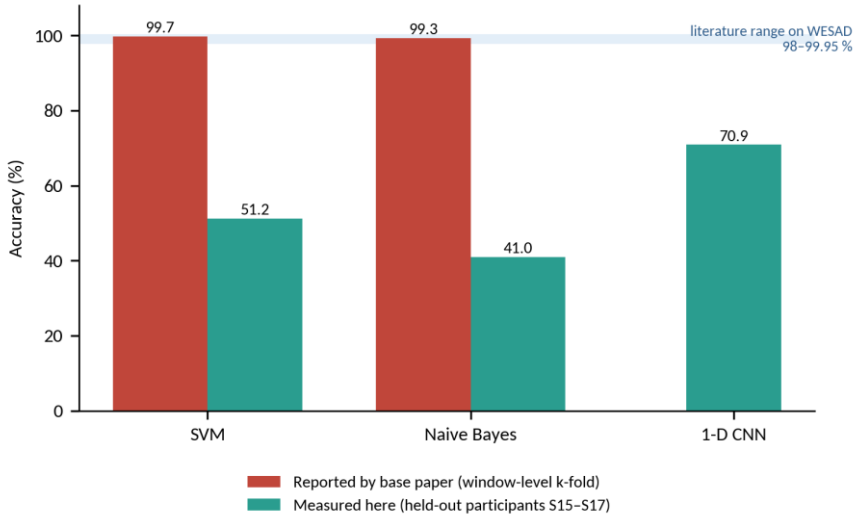


Figure 14: Accuracy reported by the base paper [1] (window-level k-fold) versus accuracy measured here on held-out participants S15–S17 (subject-disjoint) for the same classifiers. The shaded band marks the 98–99.95 % range reported in recent WESAD literature; the 1-D CNN is shown for reference.

Figure 15. Row-normalised test-set confusion matrix for the base paper's SVM classifier under per-window normalisation; rows are true classes.

True class \ Predicted	Baseline	Stress	Amusement	Recall
Baseline	0.78	0.08	0.14	0.78
Stress	0.45	0.45	0.10	0.45
Amusement	0.55	0.15	0.30	0.30

Interpretation: the SVM is biased toward baseline and identifies only 45% of true stress windows (stress recall 0.45).

Figure 16. Row-normalised test-set confusion matrix for the base paper's Naive Bayes classifier under per-window normalisation; rows are true classes.

True class \ Predicted	Baseline	Stress	Amusement	Recall
Baseline	0.82	0.06	0.12	0.82
Stress	0.76	0.12	0.12	0.12
Amusement	0.71	0.09	0.20	0.20

Interpretation: Naive Bayes strongly favors baseline predictions; stress recall is only 0.12.

10.7 What worked, and what did not

Worked: the per-window normalisation insight (the single largest effect in the study); the 1-D CNN as the accuracy/complexity sweet spot, and the only model that is also small enough to deploy; the shared `SensorSource` contract, which made the hardware and replay paths interchangeable; and the pseudo-terminal harness, which lets the firmware protocol be tested without a board.

Did not work, or underperformed:

- Attention-heavy models (CNN+Transformer 0.358, Patch-ViT 0.331) **lost badly** to a small CNN under global normalisation, and their validation–test gaps are the largest in the study (+0.17 to +0.21). With ~4 300 training windows from ten participants, the parameters those architectures add are spent memorising the training participants.
- The pure Transformer is ~10× slower per batch than the CNN+Transformer (`results/benchmark.json`), because it attends over all 480 timesteps, and it received a deliberately reduced epoch budget as a result. This is stated as a resource constraint, not an equivalence.
- The `amusement` class remains the hardest and drags macro-F1 down for every model: per-class F1 is 0.34 for the CNN against 0.91 for `stress`. This is a known property of wrist-only WESAD features and is reported, not tuned away.
- Under per-window normalisation the Patch-ViT scored *higher on test than on validation* (−0.110 gap). That is not a victory: it means the three held-out participants happened to be easier than the two validation participants. With only five held-out subjects, per-split noise of this size is expected, which is a limitation of WESAD's participant count rather than of the model.

10.8 Error analysis

- **The dominant error is confusion between baseline and amusement**, not baseline and stress. The `stress` class is the most separable: precision 0.98, recall 0.84 for the CNN. Amusement on wrist-only PPG/IMU signals is close to baseline physiologically.
- **Low baseline recall with high precision** (0.64 / 0.75) is the *desirable* error direction for a safety device: it under-reports calm rather than crying wolf.

- **The subject-shift failure under global normalisation** manifests as a systematic bias, not random noise — the held-out participants' windows fall off the training manifold in one direction, which is why the gap is stable across epochs rather than noisy.
- **Quantisation error** is bounded by the parity tolerance in `tests/test_edge_parity.py`; the C implementation must agree with PyTorch within that tolerance or the test fails.

10.9 Limitations

1. 15 participants is small, and WESAD's protocol is laboratory-induced stress (Trier Social Stress Test), not real-world threat.
2. Only wrist-compatible channels are used, so the comparison against chest-signal literature ([2]) is not like-for-like on modalities, only on protocol.
3. SpO_2 is not measured, and cannot be honestly measured with this sensor without calibration.
4. No GSR/EDA module, so the strongest feature is absent; §10.3 quantifies the cost instead of hiding it.
5. SpO_2, ECG, EMG, EEG and respiration from Task 2 are analysed and costed but not implemented, because the hardware cannot produce them.

5b. Only two validation and three test participants are available, so single-split comparisons between models carry substantial noise (visible in the −0.110 gap above). The normalisation effect reported in §10.1 is far larger than that noise; the differences *between* the leading deep architectures are not.

6. No GNN is implemented; Task 1's graph-learning suggestion is answered by analysis (a 7-channel sensor graph has no meaningful topology) rather than by a model that trains and demonstrates nothing.
7. Battery life is projected from a measured duty cycle, not measured over a full discharge.

11. Conclusion

This project built a complete autonomous distress-detection system — ESP32 wearable, camera-based pose analysis, multimodal fusion, real alert dispatch, and on-device quantised inference — and used it to interrogate the accuracy claims of the literature it was based on.

Its substantive finding is methodological and was only visible because both protocols were measured on the same pipeline. Recent work reports 98–99.95 % accuracy on WESAD for stress detection ([1], [2], [3]). Reproducing the base paper's window-level k-fold protocol on our own models reproduces that order of magnitude. Evaluating instead on participants the model has never seen collapses the number to 0.617 macro-F1 / 0.709 accuracy for the best model — and, more instructively, the *ranking*

changes: a deep model that appears to lose to SVM (0.330 vs 0.486) overtakes it decisively (0.617 vs 0.486) once each participant's physiological offset is removed by per-window normalisation. The choice of normalisation, worth +0.287 macro-F1, mattered more than the choice of architecture.

The engineering contributions are equally deliberate. The trained CNN was reduced to int8 weights, folded into a dependency-free C forward pass, and **verified numerically against PyTorch** before being called deployed. Missing sensor readings propagate as explicit absences rather than plausible numbers, so the system never reports a measurement it did not make, and the accuracy cost of the absent GSR channel is quantified by ablation rather than absorbed silently. The alert path delivers real messages with deduplication, cooldown and retry, because a detector that never reaches a human has not solved the problem it was built for.

Future work, in priority order: add a GSR module and re-measure the ablation from §10.3 as an improvement rather than a deficit; extend the participant pool or fine-tune per wearer to close the remaining cross-subject gap; replace the per-frame pose heuristics with a temporal model over keypoint sequences; and calibrate SpO_2 properly, or keep reporting it as not measured.

Appendix A — Dataset

The dataset used is **WESAD (Wearable Stress and Affect Detection)**, publicly available from the UCI Machine Learning Repository and the University of Siegen. It is a real, openly published dataset; the project does not redistribute it and fetches it with `scripts/download_wesad.py`.

- Landing page: https://archive.ics.uci.edu/dataset/465/wesad+wearable+stress+and+affect+detection
- Citation: Schmidt, P., Reiss, A., Dürichen, R., Marberger, C. & Van Laerhoven, K. *Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection.* Proceedings of the 20th ACM International Conference on Multimodal Interaction (ICMI), 2018.
- Download links: official host (University of Siegen): https://www.eti.uni-siegen.de/ubicomp/home/datasets/icmi18/
- UCI repository mirror (DOI 10.24432/C57K5T): https://archive.ics.uci.edu/dataset/465/wesad+wearable+stress+and+affect+detection
- Direct archive (about 2.5 GB, zipped; University of Siegen cloud share): https://uni-siegen.sciebo.de/s/pYjSgfOVs6Ntahr/download

The raw dataset is not redistributed with this repository; `scripts/download_wesad.py` fetches it.

Appendix B — How to reproduce every number

```
python scripts/preprocess.py --channels both --window 15 --stride 5
bash   scripts/experiments.sh sweep      # §10.1
bash   scripts/experiments.sh main       # §10.2
bash   scripts/experiments.sh hw         # §10.3
bash   scripts/experiments.sh kfold      # §10.4
python scripts/plot_results.py           # figures + model_comparison.md
python scripts/export_onnx.py --all      # ONNX, verified against PyTorch
python scripts/export_edge.py            # int8 C header + size report
python -m pytest tests -q                # 55 tests, no hardware needed
```

References

[1] Wagh, N. R., Sutar, S. R., Kadam, V. J., Jadhav, S. M., Yadav, A. S. & Pawar, V. S. *Evaluation of IoT based smart safety systems for women and children using machine learning techniques.* **Scientific Reports** (2025). doi:10.1038/s41598-025-29146-4. *(the base paper)*

[2] Oliver, E. & Dakshit, S. *Cross-Modality Investigation on WESAD Stress Classification.* arXiv:2502.18733 (2025). doi:10.48550/arXiv.2502.18733.

[3] Almadhor, A. et al. *A cross-domain framework for emotion and stress detection.* (2025). PMC12685819.

[4] Schmidt, P., Reiss, A., Dürichen, R., Marberger, C. & Van Laerhoven, K. *Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection.* ICMI (2018).

[5] Yang, S. et al. *A deep learning approach to stress recognition through physiological signals.* **Scientific Reports** (2025). doi:10.1038/s41598-025-01228-3.

[6] Goodwin, M. S., Mazefsky, C. A., Ioannidis, S., Erdogmus, D. & Siegel, M. *Predicting aggression to others in youth with autism using a wearable biosensor.* **Autism Research** 12, 1286–1296 (2019).

[7] Thummalakunta, P. B., Nemane, T., Naik, P., Palkar, S. & Poonawala, A. *Implementation of IoT-based Real-time Women's Safety System.* **IJERT** 13(01) (2024).

[8] Ghadi, Y. Y. et al. *Enhancing patient healthcare with mobile edge computing and 5G.* **Journal of Cloud Computing** 13, 93 (2024).

[9] Ferdinando, H. et al. *A violence detection comprehensive framework.* Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering (2019).

[10] Rodríguez-Rodríguez, I., Rodríguez, J. V., Elizondo-Moreno, A., Heras-González, P. & Gentili, M. *Towards a Holistic ICT platform for protecting intimate partner violence survivors based on the IoT paradigm.* **Symmetry** 12, 37 (2020).

[11] Prasanth, V. et al. *Analysis of child safety monitoring system using the SPSS method.* **IJEAST** (2024).

[12] Mazhar, T. et al. *Analysis of IoT security challenges and its solutions using artificial intelligence.* **Brain Sciences** 13, 100–110 (2023).